

Kindle E-Ink G-Helper Remote Dashboard

Project Proposal for CLI Implementation

Author: Oakich

Target Device: Amazon Kindle (Non-Jailbroken)

Target PC: ASUS ROG Ally with RTX 3080M eGPU

Date: March 1, 2026

Executive Summary

This project transforms a stock Amazon Kindle into a dedicated e-ink control surface and monitoring dashboard for G-Helper power mode management and real-time hardware statistics. The solution leverages the Kindle's experimental web browser to display a high-contrast, touch-enabled interface that communicates with a lightweight Python server running on the host PC. The implementation requires zero jailbreaking, making it accessible and reversible.

Core Functionality:

- Touch-based power mode switching (Silent, Balanced, Turbo)
 - Real-time CPU and GPU temperature monitoring
 - Low-latency command execution via virtual function keys (F16-F18)
 - E-ink optimized UI with 5-second refresh cycles
-

Technical Architecture

System Components

Component	Technology	Purpose
Frontend	Kindle Experimental Browser	Display UI and capture touch input
Backend Server	Python Flask/FastAPI	Handle HTTP requests and relay commands
Hardware Monitor	LibreHardwareMonitor	Export sensor data as JSON
Control Interface	G-Helper + PyAutoGUI	Execute power mode switches
Network Layer	Local WiFi (HTTP)	Communication between devices

Table 1: System architecture components

Data Flow

1. User taps "Turbo" button on Kindle touchscreen
2. Kindle browser sends HTTP GET request to `http://[PC_IP]:5000/mode/turbo`
3. Python Flask server receives request
4. Server executes `pyautogui.hotkey('ctrl', 'shift', 'alt', 'f18')`
5. G-Helper detects virtual F18 key press and switches to Turbo mode
6. Server returns confirmation response to Kindle
7. UI updates button state to indicate active mode

Software Requirements

PC-Side Dependencies

Python Environment:

- Python 3.10 or newer
- Flask or FastAPI (web server framework)
- PyAutoGUI (virtual keyboard input)
- Requests library (for fetching sensor data)

System Software:

- G-Helper (already installed, must be running in background)
- LibreHardwareMonitor (for temperature monitoring)
 - Enable "Web Server" option in settings
 - Default port: 8085
 - Outputs data to `http://localhost:8085/data.json`

Kindle-Side Requirements

No additional software required. The stock Kindle experimental browser is sufficient.

Configuration Steps:

1. Disable auto-sleep using the `ds search` command
2. Connect Kindle to same WiFi network as PC
3. Navigate browser to `http://[PC_IP]:5000`

Implementation Plan

Phase 1: Backend Server Development

File: `kindle_ghelper_server.py`

Core Functionality:

- HTTP server listening on port 5000
- Route handlers for power modes: `/mode/silent`, `/mode/balanced`, `/mode/turbo`
- Virtual keypress execution via PyAutoGUI
- Temperature data fetching from LibreHardwareMonitor
- JSON API endpoint: `/get-temps` returning CPU and GPU temperatures

Key Implementation Details:

- Use `pyautogui.press('f16')`, `pyautogui.press('f17')`, `pyautogui.press('f18')` for mode switching
- Parse LibreHardwareMonitor JSON to extract specific sensor values

- Implement CORS headers to allow cross-origin requests from Kindle browser
- Add error handling for LibreHardwareMonitor connection failures

Phase 2: Frontend Interface Design

File: `kindle_dashboard.html`

UI Requirements:

- High-contrast black and white color scheme
- Large touch targets (minimum 300px wide buttons)
- Three primary buttons: SILENT, BALANCED, TURBO
- Temperature display area showing CPU and GPU values
- Auto-refresh mechanism every 5 seconds for temperature updates
- Full page reload every 60 seconds to prevent e-ink ghosting

E-Ink Optimization:

- No CSS animations or transitions (causes ghosting)
- Bold fonts (minimum 24px) for readability
- Active button state: solid black background with white text
- Inactive buttons: white background with black border
- Meta refresh tag as fallback:

Phase 3: G-Helper Hotkey Configuration

Steps to Configure Virtual Function Keys:

1. Open G-Helper application
2. Navigate to "Extra" or "Hotkeys" settings
3. For each power mode, assign virtual function key:
 - Silent Mode → F16
 - Balanced Mode → F17
 - Turbo Mode → F18
4. Test by running Python snippet: `pyautogui.press('f16')`
5. Verify G-Helper responds to virtual keypresses

Phase 4: LibreHardwareMonitor Setup

1. Download LibreHardwareMonitor from official repository
2. Launch application with administrator privileges
3. Enable "Options → Remote Web Server"
4. Verify JSON endpoint accessible at
`http://localhost:8085/data.json`
5. Identify sensor paths for CPU Package Temperature and GPU Core Temperature
6. Update Python server to parse correct JSON paths

Phase 5: Kindle Configuration and Testing

1. On Kindle home screen, open search bar
2. Type `ds` and press Enter to disable screensaver
3. Note: Device will not auto-sleep until restarted (hold power 40 seconds)
4. Connect Kindle to WiFi network
5. Open Experimental Browser (Menu → Experimental Browser)
6. Navigate to PC IP address with port (e.g., 192.168.1.15:5000)
7. Bookmark page for quick access
8. Test touch responsiveness on each button
9. Monitor for browser timeouts (should be prevented by auto-refresh)

Critical Nuances and Considerations

Non-Jailbroken Kindle Limitations

Browser Constraints:

- Limited JavaScript support (ES5 compatible, avoid ES6+ features)
- No WebSocket support (use HTTP polling instead)
- Browser bars always visible (cannot achieve true fullscreen)
- Occasional crashes after extended use (mitigated by periodic refresh)

Sleep Management:

- ~ `ds` command disables screensaver but persists until device restart
- No programmatic way to re-enable auto-sleep without restart
- Battery drain: approximately 24-48 hours continuous use with WiFi active
- Recommendation: Use Kindle as "docked" display near PC

Network Configuration

Static IP Assignment:

Assign static IP to PC to prevent Kindle from losing connection after router DHCP lease renewal.

Firewall Configuration:

Ensure Windows Firewall allows incoming connections on port 5000 for Python server.

E-Ink Display Characteristics

Refresh Behavior:

- Partial refreshes (updating temperature numbers) may cause ghosting after 10-15 cycles
- Full page refresh required periodically to clear artifacts
- Implement full refresh every 60 seconds via JavaScript `location.reload()`

Touch Accuracy:

- E-ink touchscreens have lower precision than capacitive displays
- Large buttons (300px+) recommended
- Avoid placing interactive elements near screen edges

G-Helper Integration

Virtual Function Key Behavior:

- F16-F18 keys do not physically exist on keyboards
- Windows recognizes virtual key codes when sent programmatically
- Zero conflict with game hotkeys or system shortcuts

- G-Helper must be running in background (system tray) for hotkeys to work

ASUS ROG Ally Specific Considerations:

- Ally operates as Windows handheld PC with eGPU support
- G-Helper controls both internal AMD GPU and external RTX 3080M
- Power mode changes affect both GPU switching behavior and TDP limits
- Temperature monitoring should track both internal and eGPU temperatures

Temperature Monitoring Accuracy

LibreHardwareMonitor Sensor Paths:

Sensor JSON structure varies by hardware configuration. Common paths:

- CPU Package: /AMD Ryzen Z1 Extreme/Temperatures/Package
- GPU Core (eGPU): /NVIDIA GeForce RTX 3080 Mobile/Temperatures/GPU Core
- Internal GPU: /AMD Radeon Graphics/Temperatures/GPU Core

Implementation must parse JSON tree to locate correct sensor values dynamically.

Project Deliverables

1. **kindle_ghelper_server.py** - Python Flask server with routing and hardware integration
 2. **kindle_dashboard.html** - E-ink optimized HTML/CSS/JavaScript interface
 3. **requirements.txt** - Python dependencies list for pip installation
 4. **setup_guide.md** - Step-by-step configuration instructions
 5. **troubleshooting.md** - Common issues and solutions
-

Success Criteria

- User can switch G-Helper power modes via Kindle touch within 1 second latency
 - Temperature readings update every 5 seconds with $\pm 2^{\circ}\text{C}$ accuracy
 - System operates continuously for minimum 6 hours without browser crashes
 - UI remains readable on e-ink display with minimal ghosting
 - Zero conflicts with gaming or system operations
 - Setup process completable in under 30 minutes
-

Future Enhancement Possibilities

- Fan curve control interface
 - Battery percentage display
 - GPU clock speed monitoring
 - Custom fan profiles per game
 - RGB lighting control integration
 - Multi-device support (control multiple PCs from one Kindle)
 - Integration with Home Assistant for smart home controls
-

Risk Assessment

Risk	Probability	Mitigation
Kindle browser crashes	Medium	Implement auto-refresh, bookmark page
Network disconnection	Low	Use static IP, check connection health
G-Helper not responding	Low	Validate process running, restart if needed
LibreHardwareMonitor fails	Low	Implement fallback error messages
E-ink ghosting	High	Full page refresh every 60 seconds
Touch input lag	Medium	Optimize server response time

Table 2: Risk analysis and mitigation strategies

Implementation Timeline

Estimated Total Time: 3-4 hours

- **Phase 1 (Backend):** 60 minutes - Server development and testing
- **Phase 2 (Frontend):** 45 minutes - HTML/CSS/JS interface creation
- **Phase 3 (G-Helper):** 15 minutes - Hotkey configuration
- **Phase 4 (LibreHardwareMonitor):** 20 minutes - Sensor setup and path identification
- **Phase 5 (Kindle Testing):** 60 minutes - End-to-end integration and optimization

Conclusion

This project represents a practical "vibecoding" solution that transforms an underutilized e-reader into a functional PC control surface. By leveraging existing hardware and avoiding jailbreaking requirements, the implementation remains accessible and reversible.

The combination of G-Helper's virtual function key support, LibreHardwareMonitor's JSON export, and the Kindle's experimental browser creates a unique ambient computing experience optimized for the e-ink display medium.

The system provides tangible utility for PC gaming and productivity scenarios where quick access to power mode controls enhances user experience without requiring context-switching away from full-screen applications. The addition of real-time temperature monitoring transforms the Kindle into a persistent "stats screen" that complements existing desktop setups.

Ready for Gemini CLI execution and implementation.

Kindle E-Ink G-Helper Remote Dashboard

Project Proposal for CLI Implementation

Author: Oakich

Target Device: Amazon Kindle (Non-Jailbroken)

Target PC: ASUS ROG Ally with RTX 3080M eGPU

Date: March 1, 2026

Executive Summary

This project transforms a stock Amazon Kindle into a dedicated e-ink control surface and monitoring dashboard for G-Helper power mode management and real-time hardware statistics. The solution leverages the Kindle's experimental web browser to display a high-contrast, touch-enabled interface that communicates with a lightweight Python server running on the host PC. The implementation requires zero jailbreaking, making it accessible and reversible.

Core Functionality:

- Touch-based power mode switching (Silent, Balanced, Turbo)

- Real-time CPU and GPU temperature monitoring
- Low-latency command execution via virtual function keys (F16-F18)
- E-ink optimized UI with 5-second refresh cycles

Technical Architecture

System Components

Component	Technology	Purpose
Frontend	Kindle Experimental Browser	Display UI and capture touch input
Backend Server	Python Flask/FastAPI	Handle HTTP requests and relay commands
Hardware Monitor	LibreHardwareMonitor	Export sensor data as JSON
Control Interface	G-Helper + PyAutoGUI	Execute power mode switches
Network Layer	Local WiFi (HTTP)	Communication between devices

Table 3: System architecture components

Data Flow

1. User taps "Turbo" button on Kindle touchscreen
 2. Kindle browser sends HTTP GET request to `http://[PC_IP]:5000/mode/turbo`
 3. Python Flask server receives request
 4. Server executes `pyautogui.hotkey('ctrl', 'shift', 'alt', 'f18')`
 5. G-Helper detects virtual F18 key press and switches to Turbo mode
 6. Server returns confirmation response to Kindle
 7. UI updates button state to indicate active mode
-

Software Requirements

PC-Side Dependencies

Python Environment:

- Python 3.10 or newer
- Flask or FastAPI (web server framework)
- PyAutoGUI (virtual keyboard input)
- Requests library (for fetching sensor data)

System Software:

- G-Helper (already installed, must be running in background)
- LibreHardwareMonitor (for temperature monitoring)
 - Enable "Web Server" option in settings
 - Default port: 8085
 - Outputs data to `http://localhost:8085/data.json`

Kindle-Side Requirements

No additional software required. The stock Kindle experimental browser is sufficient.

Configuration Steps:

1. Disable auto-sleep using the `ds search` command
2. Connect Kindle to same WiFi network as PC
3. Navigate browser to `http://[PC_IP]:5000`

Implementation Plan

Phase 1: Backend Server Development

File: `kindle_ghelper_server.py`

Core Functionality:

- HTTP server listening on port 5000
- Route handlers for power modes: `/mode/silent`, `/mode/balanced`, `/mode/turbo`

- Virtual keypress execution via PyAutoGUI
- Temperature data fetching from LibreHardwareMonitor
- JSON API endpoint: /get-temps returning CPU and GPU temperatures

Key Implementation Details:

- Use pyautogui.press('f16'), pyautogui.press('f17'), pyautogui.press('f18') for mode switching
- Parse LibreHardwareMonitor JSON to extract specific sensor values
- Implement CORS headers to allow cross-origin requests from Kindle browser
- Add error handling for LibreHardwareMonitor connection failures

Phase 2: Frontend Interface Design

File: kindle_dashboard.html

UI Requirements:

- High-contrast black and white color scheme
- Large touch targets (minimum 300px wide buttons)
- Three primary buttons: SILENT, BALANCED, TURBO
- Temperature display area showing CPU and GPU values
- Auto-refresh mechanism every 5 seconds for temperature updates
- Full page reload every 60 seconds to prevent e-ink ghosting

E-Ink Optimization:

- No CSS animations or transitions (causes ghosting)
- Bold fonts (minimum 24px) for readability
- Active button state: solid black background with white text
- Inactive buttons: white background with black border
- Meta refresh tag as fallback:

Phase 3: G-Helper Hotkey Configuration

Steps to Configure Virtual Function Keys:

1. Open G-Helper application
2. Navigate to "Extra" or "Hotkeys" settings
3. For each power mode, assign virtual function key:
 - Silent Mode → F16
 - Balanced Mode → F17
 - Turbo Mode → F18
4. Test by running Python snippet: `pyautogui.press('f16')`
5. Verify G-Helper responds to virtual keypresses

Phase 4: LibreHardwareMonitor Setup

1. Download LibreHardwareMonitor from official repository
2. Launch application with administrator privileges
3. Enable "Options → Remote Web Server"
4. Verify JSON endpoint accessible at `http://localhost:8085/data.json`
5. Identify sensor paths for CPU Package Temperature and GPU Core Temperature
6. Update Python server to parse correct JSON paths

Phase 5: Kindle Configuration and Testing

1. On Kindle home screen, open search bar
 2. Type `~ds` and press Enter to disable screensaver
 3. Note: Device will not auto-sleep until restarted (hold power 40 seconds)
 4. Connect Kindle to WiFi network
 5. Open Experimental Browser (Menu → Experimental Browser)
 6. Navigate to PC IP address with port (e.g., `192.168.1.15:5000`)
 7. Bookmark page for quick access
 8. Test touch responsiveness on each button
 9. Monitor for browser timeouts (should be prevented by auto-refresh)
-

Critical Nuances and Considerations

Non-Jailbroken Kindle Limitations

Browser Constraints:

- Limited JavaScript support (ES5 compatible, avoid ES6+ features)
- No WebSocket support (use HTTP polling instead)
- Browser bars always visible (cannot achieve true fullscreen)
- Occasional crashes after extended use (mitigated by periodic refresh)

Sleep Management:

- `~ ds` command disables screensaver but persists until device restart
- No programmatic way to re-enable auto-sleep without restart
- Battery drain: approximately 24-48 hours continuous use with WiFi active
- Recommendation: Use Kindle as "docked" display near PC

Network Configuration

Static IP Assignment:

Assign static IP to PC to prevent Kindle from losing connection after router DHCP lease renewal.

Firewall Configuration:

Ensure Windows Firewall allows incoming connections on port 5000 for Python server.

E-Ink Display Characteristics

Refresh Behavior:

- Partial refreshes (updating temperature numbers) may cause ghosting after 10-15 cycles
- Full page refresh required periodically to clear artifacts
- Implement full refresh every 60 seconds via JavaScript `location.reload()`

Touch Accuracy:

- E-ink touchscreens have lower precision than capacitive displays
- Large buttons (300px+) recommended
- Avoid placing interactive elements near screen edges

G-Helper Integration

Virtual Function Key Behavior:

- F16-F18 keys do not physically exist on keyboards
- Windows recognizes virtual key codes when sent programmatically
- Zero conflict with game hotkeys or system shortcuts
- G-Helper must be running in background (system tray) for hotkeys to work

ASUS ROG Ally Specific Considerations:

- Ally operates as Windows handheld PC with eGPU support
- G-Helper controls both internal AMD GPU and external RTX 3080M
- Power mode changes affect both GPU switching behavior and TDP limits
- Temperature monitoring should track both internal and eGPU temperatures

Temperature Monitoring Accuracy

LibreHardwareMonitor Sensor Paths:

Sensor JSON structure varies by hardware configuration. Common paths:

- CPU Package: /AMD Ryzen Z1 Extreme/Temperatures/Package
- GPU Core (eGPU): /NVIDIA GeForce RTX 3080 Mobile/Temperatures/GPU Core
- Internal GPU: /AMD Radeon Graphics/Temperatures/GPU Core

Implementation must parse JSON tree to locate correct sensor values dynamically.

Project Deliverables

1. **kindle_ghelper_server.py** - Python Flask server with routing and hardware integration
 2. **kindle_dashboard.html** - E-ink optimized HTML/CSS/JavaScript interface
 3. **requirements.txt** - Python dependencies list for pip installation
 4. **setup_guide.md** - Step-by-step configuration instructions
 5. **troubleshooting.md** - Common issues and solutions
-

Success Criteria

- User can switch G-Helper power modes via Kindle touch within 1 second latency
 - Temperature readings update every 5 seconds with $\pm 2^{\circ}\text{C}$ accuracy
 - System operates continuously for minimum 6 hours without browser crashes
 - UI remains readable on e-ink display with minimal ghosting
 - Zero conflicts with gaming or system operations
 - Setup process completable in under 30 minutes
-

Future Enhancement Possibilities

- Fan curve control interface
 - Battery percentage display
 - GPU clock speed monitoring
 - Custom fan profiles per game
 - RGB lighting control integration
 - Multi-device support (control multiple PCs from one Kindle)
 - Integration with Home Assistant for smart home controls
-

Risk Assessment

Risk	Probability	Mitigation
Kindle browser crashes	Medium	Implement auto-refresh, bookmark page
Network disconnection	Low	Use static IP, check connection health
G-Helper not responding	Low	Validate process running, restart if needed
LibreHardwareMonitor fails	Low	Implement fallback error messages
E-ink ghosting	High	Full page refresh every 60 seconds
Touch input lag	Medium	Optimize server response time

Table 4: Risk analysis and mitigation strategies

Implementation Timeline

Estimated Total Time: 3-4 hours

- **Phase 1 (Backend):** 60 minutes - Server development and testing
- **Phase 2 (Frontend):** 45 minutes - HTML/CSS/JS interface creation
- **Phase 3 (G-Helper):** 15 minutes - Hotkey configuration
- **Phase 4 (LibreHardwareMonitor):** 20 minutes - Sensor setup and path identification
- **Phase 5 (Kindle Testing):** 60 minutes - End-to-end integration and optimization

Conclusion

This project represents a practical "vibecoding" solution that transforms an underutilized e-reader into a functional PC control surface. By leveraging existing hardware and avoiding jailbreaking requirements, the implementation remains accessible and reversible.

The combination of G-Helper's virtual function key support, LibreHardwareMonitor's JSON export, and the Kindle's experimental browser creates a unique ambient computing experience optimized for the e-ink display medium.

The system provides tangible utility for PC gaming and productivity scenarios where quick access to power mode controls enhances user experience without requiring context-switching away from full-screen applications. The addition of real-time temperature monitoring transforms the Kindle into a persistent "stats screen" that complements existing desktop setups.

Ready for Gemini CLI execution and implementation.